

56. #include, et les onglets de la page

Le fichier voisin est versé ici, avant toute compilation.

1. Ce qu'il faut comprendre

`#include "dessins.cpp"` **verse le fichier voisin à cet endroit** avant que le compilateur ne voie quoi que ce soit. Il n'y a donc ni éditeur de liens, ni ordre de compilation à connaître.

Dans la page, les fichiers voisins sont des **onglets** : `#include` ne voit pas la différence. Et quand une faute tombe dans un fichier inclus, le message désigne **son** fichier et **sa** ligne — pas la ligne du texte assemblé, que personne n'a jamais vue.

Le programme ci-dessous tient en un fichier ; la leçon « Découper son programme » en montre un sur cinq.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1316 octets** de cartouche.

1. UNE FONCTION — DOUBLER()

```
uint8_t doubler(uint8_t n) { return n * 2; }
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

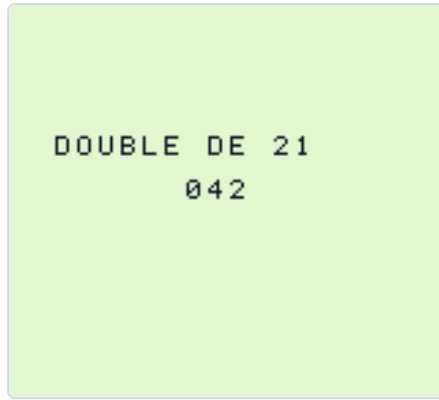
```
int main() {  
    texte(2, 6, "DOUBLE DE 21");  
    nombre(8, 8, doubler(21));  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« DOUBLE DE 21 », et 042 en dessous — la fonction a été appelée.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le calcul est juste

✓ l'étiquette est là

Ensuite : **57. Une « struct » : ce qui va ensemble** — niveau 8 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++